

Exception Handling

```
# =====
# PYTHON EXCEPTION HANDLING
# Data Engineering Practice
# =====

# TASK 1 - Basic Exception Handling
def divide_numbers(a, b):

    try:
        result = a / b
        return result

    except ZeroDivisionError:
        return "Cannot divide by zero"

print(divide_numbers(10, 2))
print(divide_numbers(10, 0))

print("=" * 50)

# TASK 2 - Handle Multiple Exceptions
def safe_divide(a, b):
    try:
        a = int(a)
        b = int(b)
        return a / b

    except ValueError:
        return "Values must be numbers"

    except ZeroDivisionError:
        return "Cannot divide by zero"

print(safe_divide("100", "10"))
print(safe_divide("100", "0"))
print(safe_divide("abc", "10"))
print("=" * 50)

# TASK 3 - Validate Numeric Amount
# Requirements:
# - Convert amount to float
# - Reject invalid amounts
# - Reject negative amounts
def validate_amount(amount):
    try:
        amount = float(amount)
        if amount < 0:
            return "Amount cannot be negative"
        return amount

    except ValueError:
        return "Invalid amount"

print(validate_amount("500"))
print(validate_amount("1250.50"))
print(validate_amount("-100"))
print(validate_amount("abc"))

print("=" * 50)
```

```

# TASK 4 - Exception Handling in ETL
#
# Requirements:
# - Extract order_id
# - Convert amount to float
# - Validate amount
# - Calculate tax
# - Calculate total
# - Handle invalid data

def transform_order(order):

    try:

        # Extract
        order_id = order["order_id"]
        amount = float(order["amount"])

        # Validate
        if amount < 0:
            raise ValueError("Negative amount")

        # Transform
        tax = amount * 0.14
        total = amount + tax

        # Return transformed record
        return {
            "order_id": order_id,
            "amount": amount,
            "tax": tax,
            "total": total
        }

    except KeyError as e:
        return {
            "status": "REJECTED",
            "error": f"Missing field: {e}"
        }

    except ValueError as e:
        return {
            "status": "REJECTED",
            "error": str(e)
        }

orders = [
    {"order_id": 1, "amount": "500"},
    {"order_id": 2, "amount": "abc"},
    {"order_id": 3, "amount": "-100"},
    {"order_id": 4, "amount": "750"}
]

print("\nTASK 4")

for order in orders:
    result = transform_order(order)
    print(result)

print("=" * 50)

```

```

# EXAMPLE - Real Data Engineering Exception Handling
# This example:
# - Extracts data
# - Validates data
# - Transforms valid records
# - Rejects bad records
# - Keeps the pipeline running

def transform_order_etl(order):
    try:

        # Extract
        order_id = order["order_id"]
        amount = float(order["amount"])

        # Validate
        if amount < 0:
            raise ValueError("Amount cannot be negative")

        # Transform
        tax = amount * 0.14
        total = amount + tax

        # Return valid record
        return {
            "order_id": order_id,
            "amount": amount,
            "tax": tax,
            "total": total,
            "status": "VALID"
        }

    # Handle missing fields
    except KeyError as e:

        return {
            "order_id": order.get("order_id", "UNKNOWN"),
            "status": "REJECTED",
            "error": f"Missing field: {e}"
        }

    # Handle invalid values
    except ValueError as e:

        return {
            "order_id": order.get("order_id", "UNKNOWN"),
            "status": "REJECTED",
            "error": str(e)
        }

# INPUT DATA
orders = [
    {"order_id": 1, "amount": "500"},
    {"order_id": 2, "amount": "1200"},
    {"order_id": 3, "amount": "abc"},
    {"order_id": 4, "amount": "-100"},
    {"order_id": 5}
]

# PROCESS DATA
print("\nETL EXAMPLE")

for order in orders:

    result = transform_order_etl(order)

    print(result)a

```